

Stack vs Heap Memory in Java

how the **JVM** divides memory into **Stack** and **Heap**,
allowing applications to run optimally



```
int subscribers;
```

```
Channel channel = new Geekific();
```

```
likeAndSubscribe();
```

```
String str = "Like";
```

memory

heap
stack

The diagram illustrates memory allocation for the provided code. A large yellow oval represents the memory space, which is divided into two sections: the 'stack' (bottom) and the 'heap' (top). The labels 'stack' and 'heap' are written in red, italicized font. Two curved arrows indicate the allocation of variables: one arrow points from the 'new' keyword in the second line of code to the 'heap' section, and another arrow points from the 'str' variable in the fourth line of code to the 'stack' section.



```
public static Channel buildChannel(int id, String name) {  
    return new Channel(id, name);  
}
```

```
public static class Channel {  
    int id;  
    String name;  
  
    public Channel(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```



```
public static void main(String[] args) {  
    int id = 1;  
    String name = "Geekific";  
    Channel channel = null;  
    channel = buildChannel(id, name);  
}
```



main

Call Stack

stores information about
the active sub-routines of
a computer program

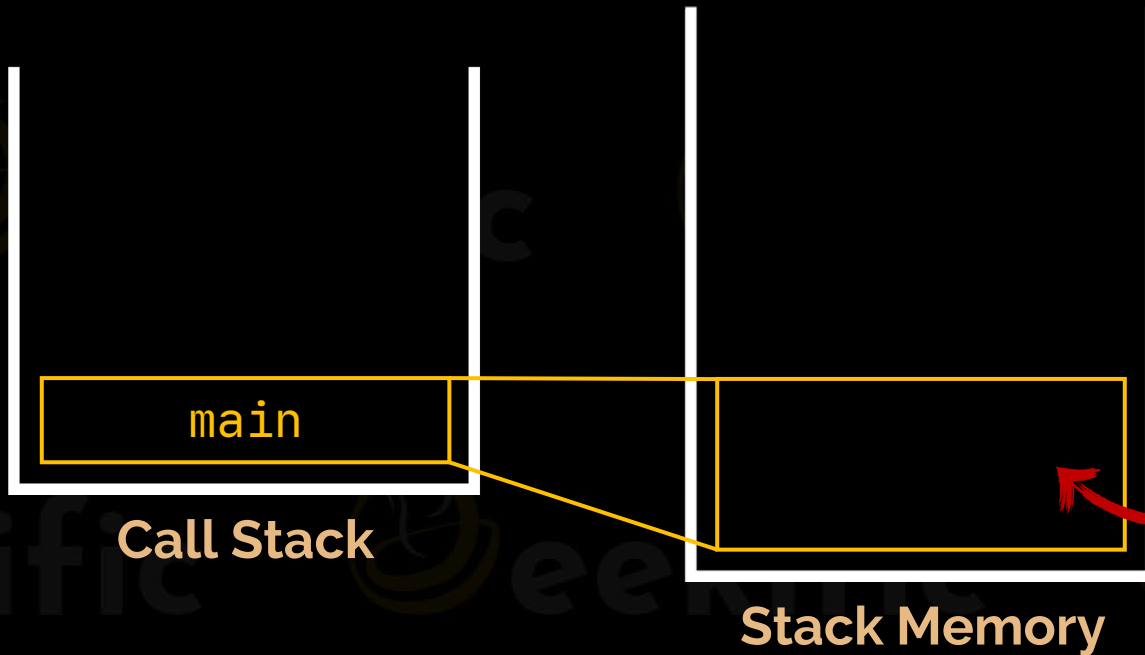


```
public static Channel buildChannel(int id, String name) {  
    return new Channel(id, name);  
}
```

```
public static class Channel {  
    int id;  
    String name;  
  
    public Channel(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```



```
public static void main(String[] args) {  
    int id = 1;  
    String name = "Geekific";  
    Channel channel = null;  
    channel = buildChannel(id, name);  
}
```



stores variables values
and references to objects
stored in the Heap used
by the method



Primitives vs References

primitive types like int have their value stored **directly in the Stack**

reference types like String and Objects have a **reference** in the Stack that **points to the Heap**

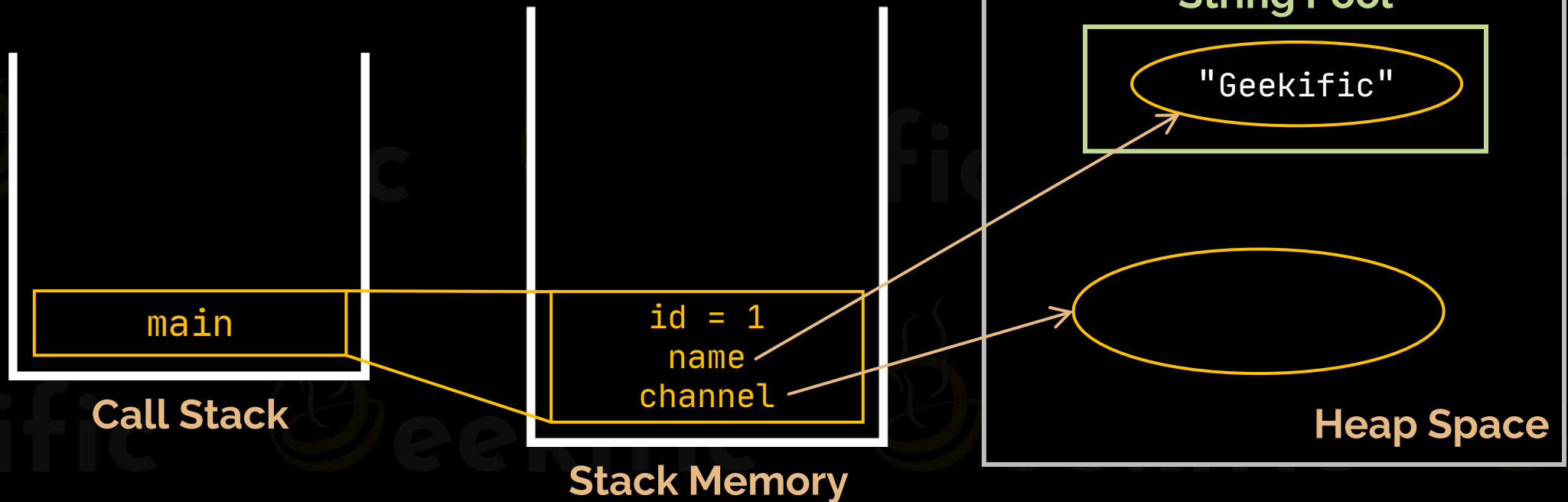
if the reference is **null**, it points to nothing in the Heap



```
public static Channel buildChannel(int id, String name) {  
    return new Channel(id, name);  
}
```

```
public static class Channel {  
    int id;  
    String name;  
  
    public Channel(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

```
public static void main(String[] args) {  
    int id = 1;  
    String name = "Geekific";  
    Channel channel = null;  
    channel = buildChannel(id, name);  
}
```



The String Pool

string allocation, like all object allocation, is costly in both space and time

to reduce the number of String objects, the JVM keeps a pool of strings in the Heap

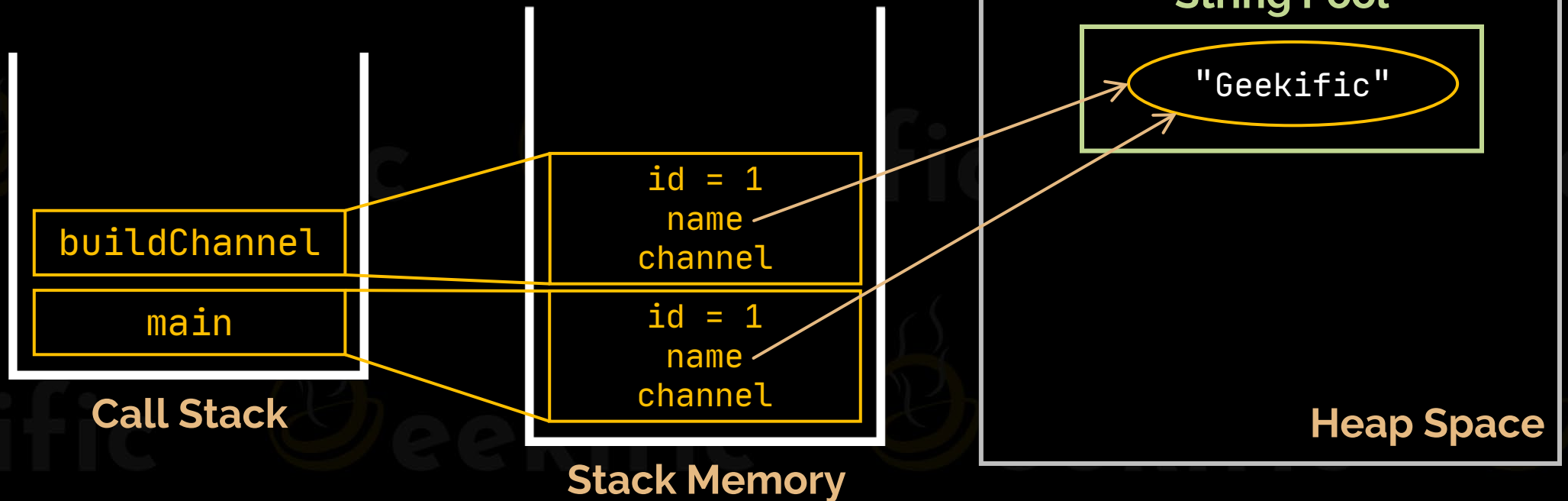
two variables with the same string literal will point to the same object in the pool



```
public static Channel buildChannel(int id, String name) {  
    return new Channel(id, name);  
}
```

```
public static class Channel {  
    int id;  
    String name;  
  
    public Channel(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

```
public static void main(String[] args) {  
    int id = 1;  
    String name = "Geekific";  
    Channel channel = null;  
    channel = buildChannel(id, name);  
}
```



How Method Calls Use Memory

every method call is pushed onto the **Call Stack** and gets its own block in **Stack Memory** – **id**'s value is copied directly, **name**'s reference points to the **String Pool**

the **new** operator pushes the **constructor** onto the Call Stack and creates the actual object in the **Heap**

the constructor's stack frame stores its attributes and a **this** reference pointing to the object in the Heap

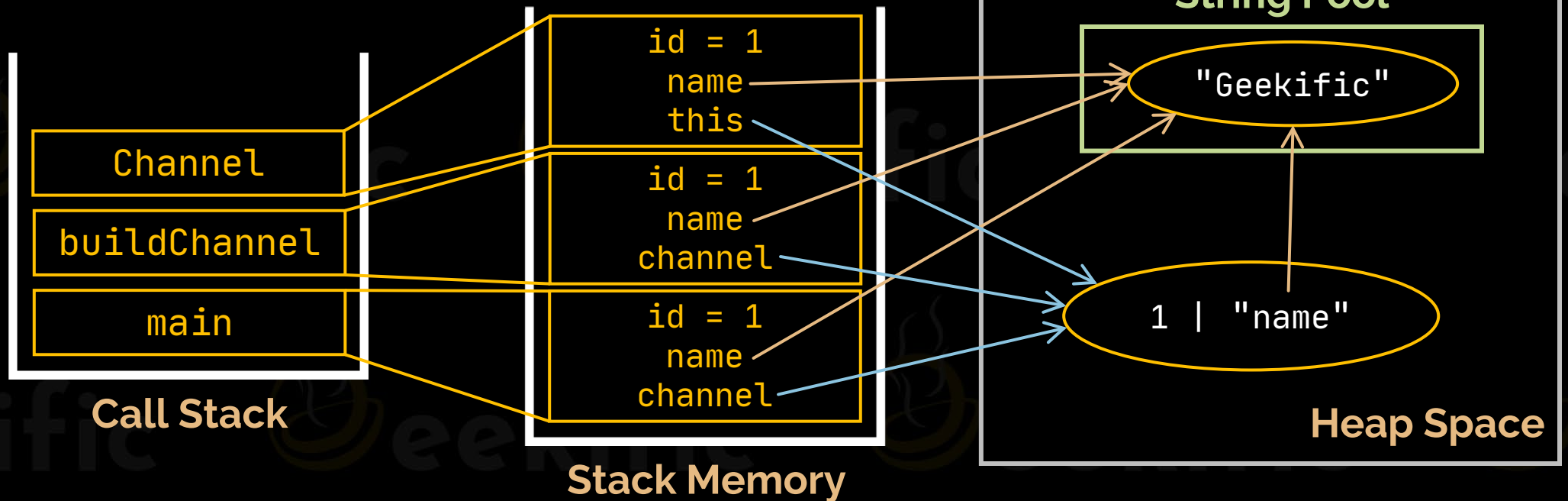


```
public static Channel buildChannel(int id, String name) {  
    return new Channel(id, name);  
}
```

```
public static class Channel {  
    int id;  
    String name;  
  
    public Channel(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

`new Channel()` pushes the constructor – `id`, `name`, and `this` are stored in its stack frame

```
public static void main(String[] args) {  
    int id = 1;  
    String name = "Geekific";  
    Channel channel = null;  
    channel = buildChannel(id, name);  
}
```

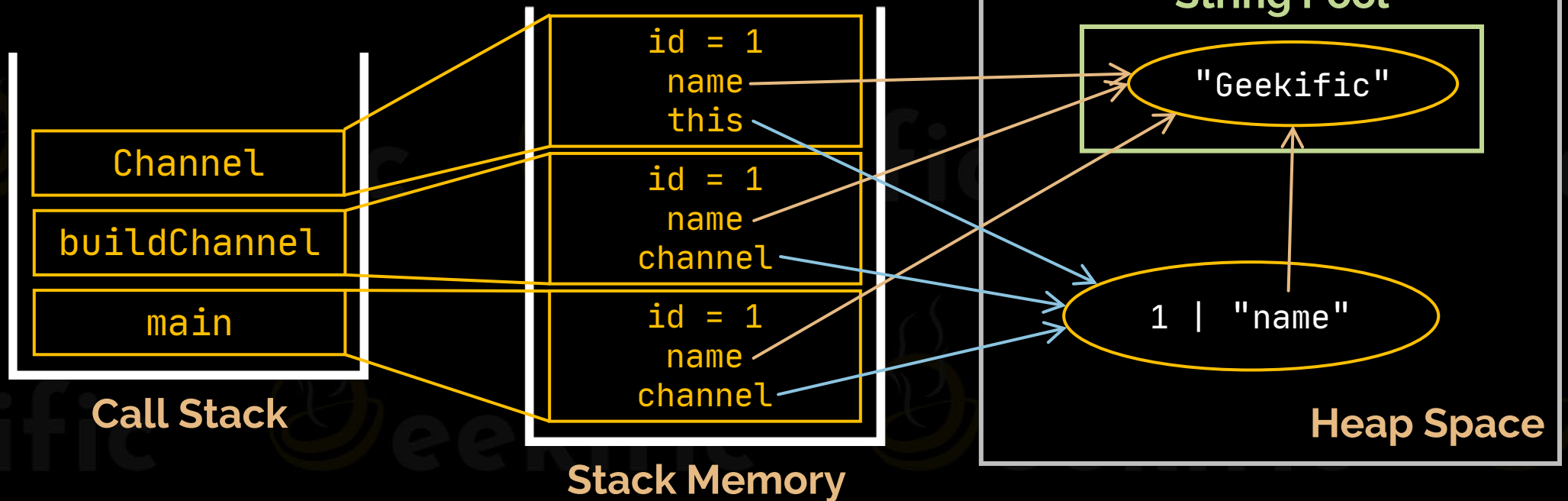


```
public static Channel buildChannel(int id, String name) {  
    return new Channel(id, name);  
}
```

```
public static class Channel {  
    int id;  
    String name;  
  
    public Channel(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

the Channel object is created in the Heap – all references now point to it

```
public static void main(String[] args) {  
    int id = 1;  
    String name = "Geekific";  
    Channel channel = null;  
    channel = buildChannel(id, name);  
}
```

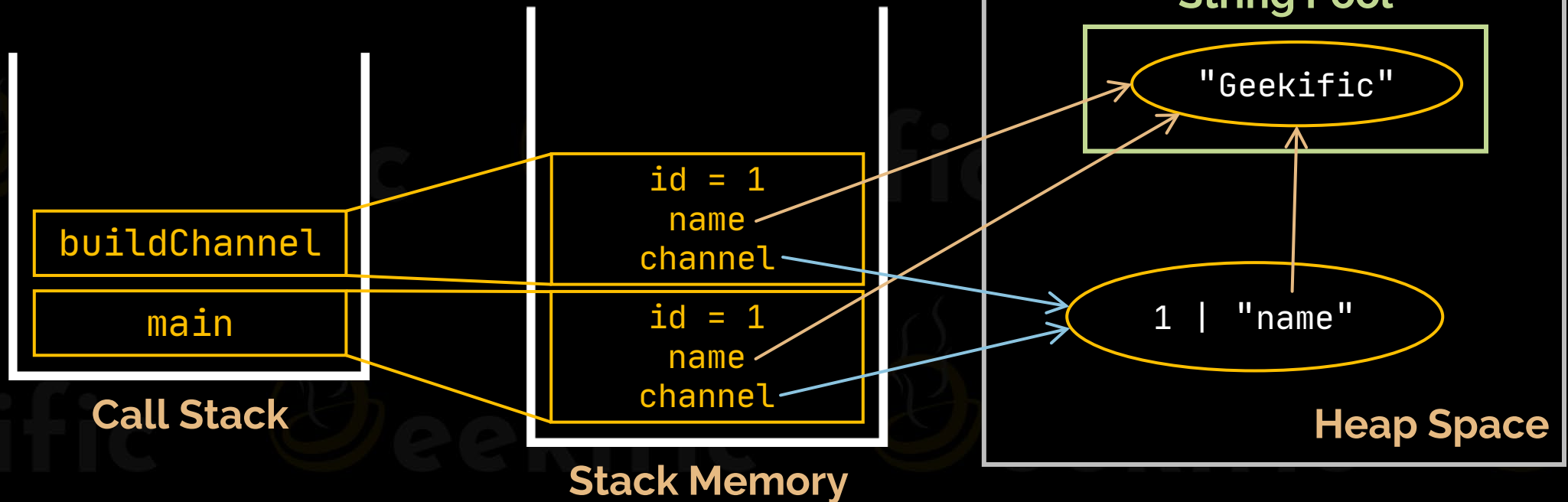


```
public static Channel buildChannel(int id, String name) {  
    return new Channel(id, name);  
}
```

```
public static class Channel {  
    int id;  
    String name;  
  
    public Channel(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

constructor is popped – buildChannel now references the Channel object in the Heap

```
public static void main(String[] args) {  
    int id = 1;  
    String name = "Geekific";  
    Channel channel = null;  
    channel = buildChannel(id, name);  
}
```

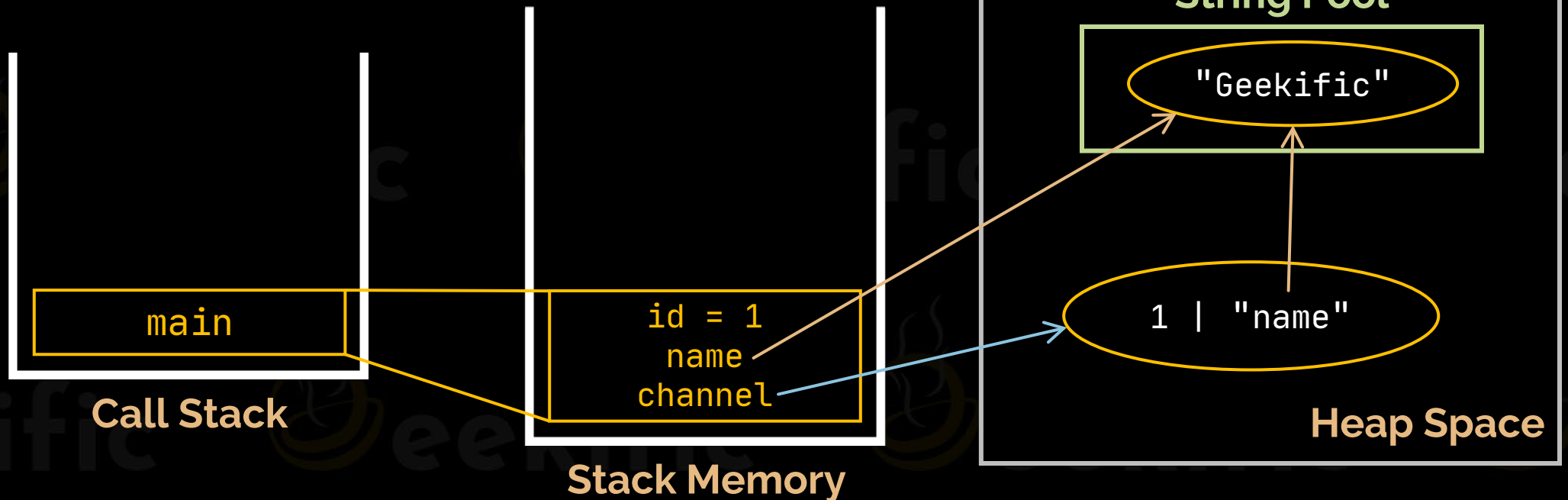


```
public static Channel buildChannel(int id, String name) {  
    return new Channel(id, name);  
}
```

```
public static class Channel {  
    int id;  
    String name;  
  
    public Channel(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

buildChannel is popped – main's channel reference now points to the actual object

```
public static void main(String[] args) {  
    int id = 1;  
    String name = "Geekific";  
    Channel channel = null;  
    channel = buildChannel(id, name);  
}
```



Returning from Methods

when the **constructor** finishes, its stack frame is **popped** and **buildChannel** now references the correct Channel object

when **buildChannel** returns, it is also popped and **main's null** reference is replaced with the actual object

references propagate back through the call chain because the **Heap object persists** even as stack frames come and go



Memory Cleanup

when a method finishes, its stack frame is automatically popped in last-in-first-out order

objects in the Heap are only cleaned by the Garbage Collector when no references point to them

the Heap is used by the entire application, while the Stack is used one method at a time

only after main exits are its Heap references eligible for cleanup



Stack

used in parts, one at a time

stores primitive type variables
and references to objects that
are created in Heap

is automatically allocated and
deallocated when a method is
called and returned

provides a high-speed access to
everything stored inside it

will never be fragmented

Heap

used by the entire application

is allocated when new objects
are created, and deallocated by
the Garbage Collector

there's no enforced pattern to
the allocation and deallocation

time to access the variables
much slower than the Stack

contrary to the Stack, there is no
size limit on the Heap



Key Takeaways

Stack – fast, automatic, LIFO,
never fragmented, but **fixed size**

Heap – no size limit, used by the entire app,
but **slower access** and can **fragment**

String Pool – reuses literals to save memory

Garbage Collector – frees Heap objects with no references

